

Competitive Programming

Section 14: Core Patterns Mixed Contest

Competitive Programming Course

2026-07-05

Table of contents

1	Why a mixed contest now	3
2	Phase 1 pattern map	4
3	Contest triage	5
4	Problem A signal: complete window	6
5	Problem B signal: minimum split limit	7
6	Problem C signal: active colors	8
7	Mixed-problem thinking	9
8	Complexity targets	10
9	Common contest mistakes	11
9.1	Solving the first problem first automatically	11
9.2	Seeing a familiar word but missing a constraint	11
9.3	Forgetting endpoint conventions	11
9.4	Printing sorted-query order	11
10	Debugging mixed sets	12
11	C++ implementation notes	13
12	Python implementation notes	14
13	Submission checklist	15
14	Exercises	16
14.1	A. Complete Window	16
14.2	B. Minimum Split Limit	16
14.3	C. Active Colors	16

1 Why a mixed contest now

Sections 8 through 13 taught the core medium-level patterns:

- binary search on the answer;
- greedy exchange and constructive invariants;
- sweep line over events;
- monotonic stacks and dequeues;
- offline processing and coordinate compression.

Real contests do not label problems. This section trains the missing step:

Which pattern is this statement asking for?

The local set should be attempted like a short contest. Read all three problems first, estimate the intended complexity, and choose the order deliberately.

2 Phase 1 pattern map

Statement signal	Likely pattern
shortest/longest contiguous segment with maintainable counts	two pointers / sliding window
minimize the maximum feasible value	binary search on answer
intervals, active objects, point queries	sweep line
next greater/smaller, window maximum	stack or deque
queries can be sorted without changing meaning	offline processing
large values only need equality/order	coordinate compression or maps
local choice needs proof	greedy exchange
construct lexicographically smallest valid object	greedy with suffix feasibility

This table is not a replacement for proof. It is a triage tool.

3 Contest triage

Do a fast first pass:

```
for every problem:
    read the statement
    read constraints
    identify output type
    list one or two candidate patterns
    mark confidence: high / medium / low
```

Then solve in an order that maximizes reliable progress.

Questions to ask:

```
Is the answer a value or an object?
Are there many queries?
Can queries be reordered?
Does a pointer only move forward?
Is there a monotone yes/no predicate?
Do endpoints or ties matter?
```

4 Problem A signal: complete window

Problem A asks for the shortest contiguous subarray containing all labels $1..m$.

Statement clues:

- contiguous subarray;
- maintain counts of labels;
- shortest valid window;
- adding right endpoint can only add labels;
- removing left endpoint can only remove labels.

That points to sliding window with a frequency table.

The invariant:

```
freq[label] counts occurrences inside the current window [left, right].  
have is the number of required labels with positive frequency.
```

When `have == m`, the window is valid, so shrink from the left while it remains valid.

5 Problem B signal: minimum split limit

Problem B asks:

```
Split into at most d groups.  
Minimize the maximum group sum.
```

This is a classic binary-search-on-answer shape.

Candidate answer:

```
C = allowed maximum group sum
```

Feasibility predicate:

```
Can we split the array into at most d groups where every group sum <= C?
```

If capacity C works, every larger capacity also works. That monotonicity gives binary search.

The check is greedy:

```
scan left to right  
fill the current group until adding the next item would exceed C  
then start a new group
```

This uses the fewest groups for that capacity because ending a group earlier cannot make future groups larger-capacity.

6 Problem C signal: active colors

Problem C has colored half-open intervals and point queries:

```
How many distinct colors are active at x?
```

Statement clues:

- intervals;
- active at a point;
- queries;
- original query order;
- half-open endpoint convention.

That points to sweep line with answer restoration.

Events:

```
end event at r  
start event at l  
query event at x
```

Tie order for $[l, r)$:

```
end -> start -> query
```

Why?

- an interval ending at x is not active at x ;
- an interval starting at x is active at x ;
- a query at x should see the state after both facts are applied.

State:

```
freq[color] = number of active intervals of this color  
distinct = number of colors with positive frequency
```

7 Mixed-problem thinking

Many contest tasks combine patterns.

Problem C combines:

- sweep line from Section 11;
- maps/frequencies from Section 3;
- answer restoration from Section 13.

This does not make it a new algorithm. It means you should decompose the statement:

```
intervals -> sweep  
distinct colors -> frequency map  
queries out of order -> original ids
```

The strongest solvers do this calmly. They do not try to name the whole problem with one tag.

8 Complexity targets

For $n, q \leq 200000$, aim for:

```
O(n log n)
O((n+q) log(n+q))
O(n)
```

Avoid:

```
O(nq)
O(n^2)
iterating over coordinate ranges
```

For each local problem:

Problem	Intended complexity
Complete Window	$O(n)$
Minimum Split Limit	$O(n \log \text{sum})$
Active Colors	$O((n+q) \log(n+q))$

9 Common contest mistakes

9.1 Solving the first problem first automatically

First in statement order is not always easiest for you. Read the whole set.

9.2 Seeing a familiar word but missing a constraint

For example, “window” is not always two pointers. If negative values affect a sum, two pointers may fail. In this local Problem A, counts are monotone under adding/removing endpoints, so two pointers works.

9.3 Forgetting endpoint conventions

For half-open intervals, $[1, 5)$ is inactive at 5.

Problem C is designed to punish the wrong tie order.

9.4 Printing sorted-query order

Any time you sort queries, attach original ids.

10 Debugging mixed sets

When a solution fails:

1. Is the chosen pattern valid?
2. Is the invariant true after every update?
3. Are boundary cases correct?
4. Is output order correct?
5. Can a brute-force oracle test small random cases?

For this section, the local judge uses brute-force generators for all three problems:

- all subarrays for Problem A;
- all splits by dynamic programming for Problem B;
- direct interval scanning for Problem C.

11 C++ implementation notes

Use `long long` for sums and capacities.

For event sorting, define a small struct:

```
struct Event {  
    long long x;  
    int kind;  
    long long color;  
    int idx;  
};
```

For color frequencies, `unordered_map<long long,int>` is sufficient here.

12 Python implementation notes

Use lists for dense label frequencies in Problem A:

```
freq = [0] * (m + 1)
```

Use dictionaries for sparse colors in Problem C:

```
freq[color] = freq.get(color, 0) + 1
```

For binary search, Python integers safely handle large sums.

13 Submission checklist

- Did you identify the pattern before coding?
- Did you write the invariant or predicate in one sentence?
- Are all sums wide enough?
- Are interval endpoints handled with the correct tie order?
- Are query answers restored to original order?
- Did you test impossible and boundary cases?

14 Exercises

14.1 A. Complete Window

Sliding window plus frequency counts.

14.2 B. Minimum Split Limit

Binary search on the answer plus greedy feasibility.

14.3 C. Active Colors

Sweep line plus color frequencies plus answer restoration.